DIGITAL DESIGN

LAB9 UART , DEBUG

WANGW6@SUSTECH.EDU.CN

TOPIC

- 1. Verilog
  - parameter used at module level
- 2. Uart demo
- 3. Debug by simulation

WANGW6@SUSTECH.EDU.CN

PARAMETER USED AT MODULE LEVEL

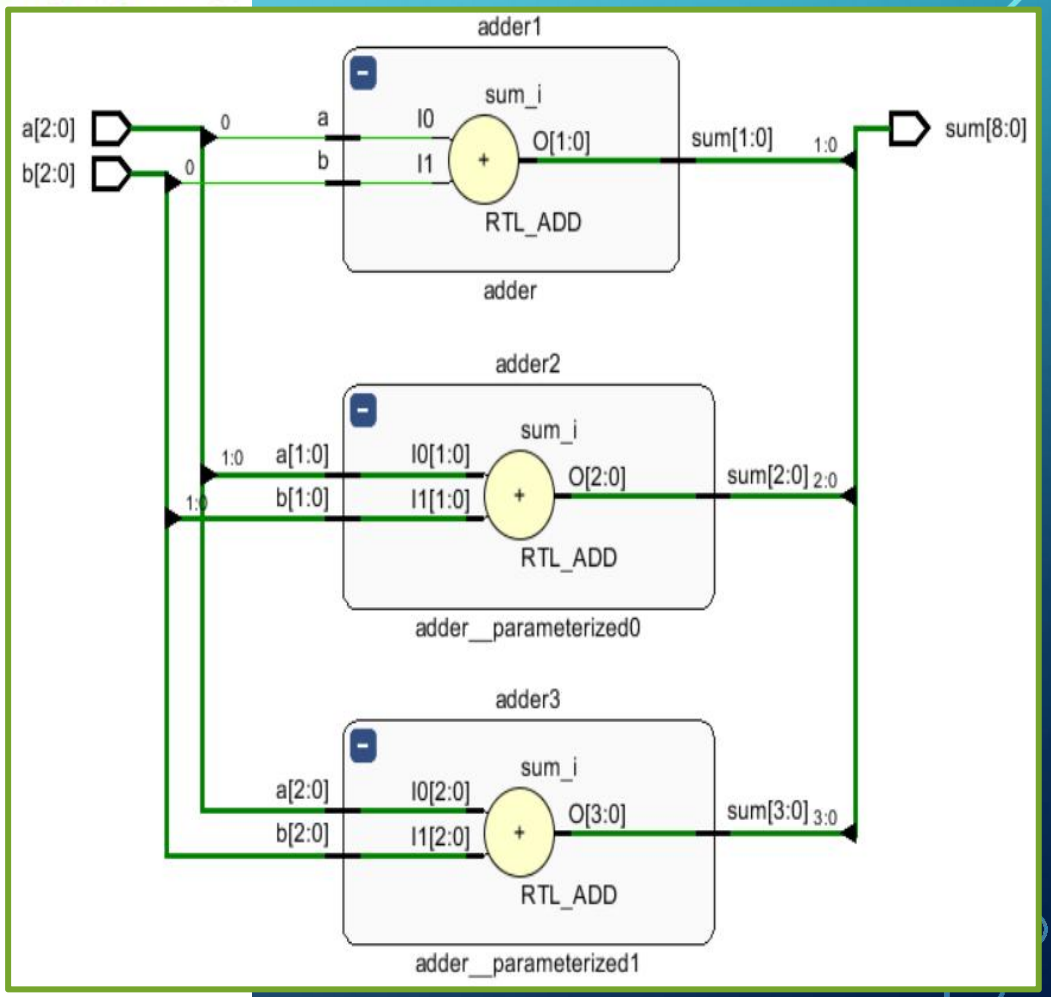
In Verilog, the **parameter** keyword is used to define compile-time constants that enhance code reusability and maintainability.

Parameters are declared with parameter and assigned constant values.

Here is an example about how parameters used at the module level.

```
module adder #(parameter WIDTH = 2) (
   input [WIDTH-1:0] a,
   input [WIDTH-1:0] b,
   output [WIDTH:0] sum
);
   assign sum = a + b;
endmodule
```

```
module para_adder( input [2:0] a, b, output [8:0] sum  );

   adder #(.WIDTH(1)) adder1 (
      .a(a[0]),
      .b(b[0]),
      .sum(sum[1:0])
   );

   adder adder2 (
         .a(a[1:0]),
         .b(b[1:0]),
         .sum(sum[4:2])
      );

   adder #(.WIDTH(3))adder3 (
         .a(a[2:0]),
         .b(b[2:0]),
         .sum(sum[8:5])
            );
endmodule
```



WANGW6@SUSTECH.EDU.CN

TOPIC

- 1. Verilog
  - parameter used at module level
- **2. Uart demo**
- 3. Debug by simulation

WANGW6@SUSTECH.EDU.CN

UART

UART (Universal Asynchronous Receiver Transmitter) is a widely used serial communication protocol, whose core feature is to transmit data asynchronously without the need for clock line synchronization.

The sender converts parallel data into a serial signal, and the receiver samples the data line at the baud rate.

The start bit triggers the receiver to start sampling, while the stop bit provides an opportunity for clock synchronization.

The UART data frame consists of the following parts (arranged in transmission order)：Starting bit, Data bits,Parity check (optional) and Stop bit. When no data is transmitted, the data line remains at a high level (1) which means in Idle state.

key parameters

Baud rate: Both parties need to agree in advance (such as 9600, 115200), with an allowable error of ≤ 10%

Full duplex communication: only requires three wires: TX (transmit), RX (receive), and GND (ground)

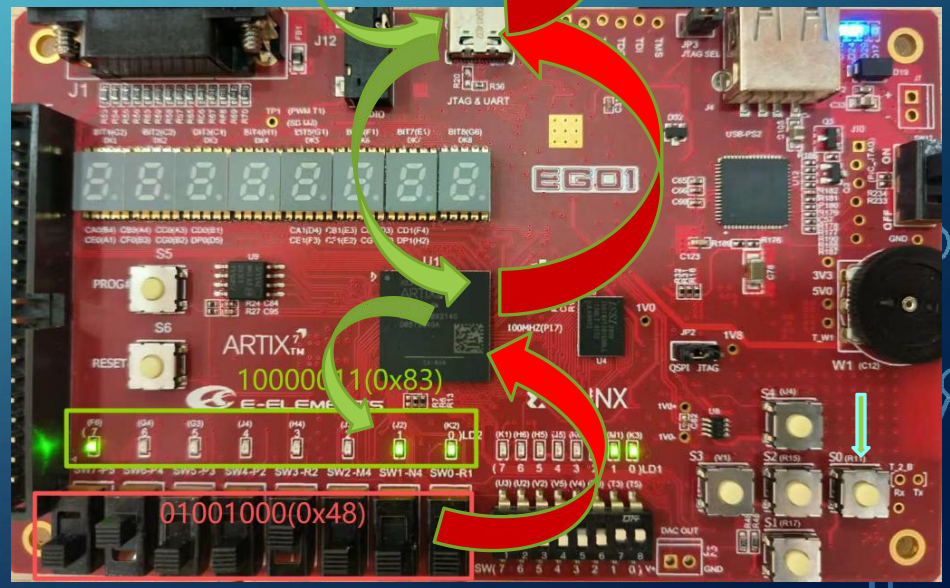
WANGW6@SUSTECH.EDU.CN

UART FRAME STRUCTURE DIAGRAM

- **Start Bit**:
  - Representation: A single bit at a logic level **0 (low)**.
  - Purpose: **Indicates the beginning of a frame**. It helps the receiver synchronize its clock with the sender.
- **Data Bits**:
  - Representation: **A sequence of bits, typically 5 to 9 bits long. Most commonly, 8 bits are used**.
  - Purpose: Carries the actual data being transmitted. **The least significant bit (LSB) is usually sent first**.
- Parity Bit (Optional):
  - Representation: A single bit that can be either 0 or 1, depending on the parity chosen.
  - Types: None, Even Parity, Odd Parity
  - Purpose: Provides basic error detection.
- **Stop Bits**:
  - Representation: **One or more bits at a logic level 1 (high)**. Common configurations are 1, 1.5, or 2 stop bits.
  - Purpose: **Indicates the end of the frame** and provides a buffer for clock synchronization between the sender and receiver.

WANGW6@SUSTECH.EDU.CN

UART DEMO (1) -FUNCTION



The **circuit** works on the FPGA chip of the **EGO1**, communicates with the **UART communication software(串口调试助手)** running on the PC through the UART port.

- The dual transmission agreement uses **115200** as the **baud rate** and **does not use parity bits during communication**. **The start and end bits are both 1 bit, and the data bits are 8 bits**.

start bit(1)	data bits(8)	stop bit(1)

- Use the **8 dip switches on the EGO1** to set the sending data, after be comfirmed to send by pressing the confirm key, the data is sent to **UART communication software(串口调试助手)** and displayed on its **receive window(串口数据接收)**.

- The data specified in the sending window of **UART communication software(串口调试助手)** is sent to the circuit and be displayed on 8 LED lights on EGO1.

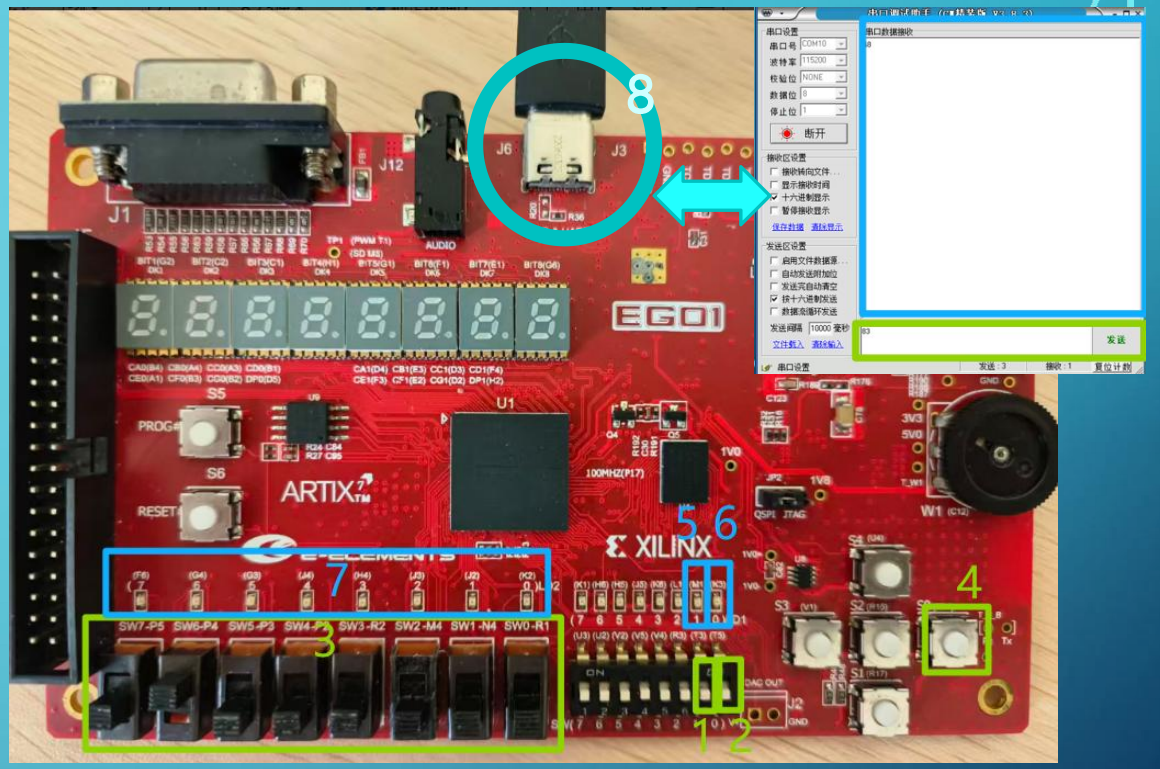
WANGW6@SUSTECH.EDU.CN

UART DEMO (2) - I/O

INPUT:
1. set Uart tx work or not
2. set Uart rx work or not
3. set data which would be sent on UART TX port
4. when pressed, send the data on the switches on UART port

OUTPUT:
5. show uart tx work or not
6. show uart rx work or not
7. display the data received on UART RX

INPUT / OUTPUT :
8. UART RX / UART TX



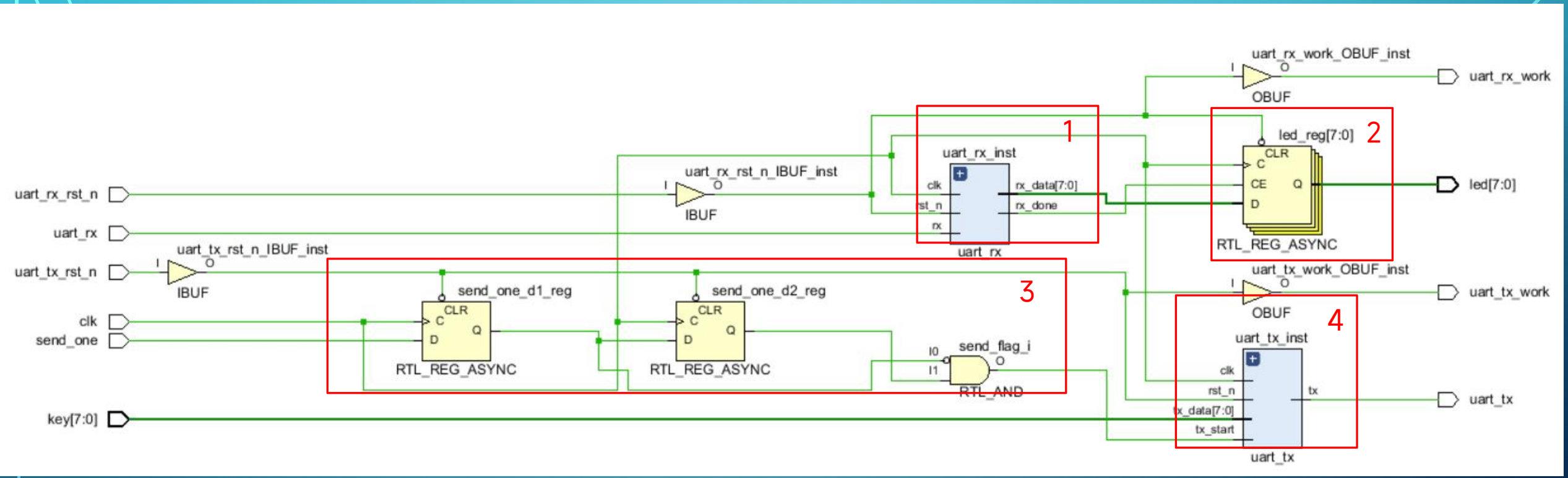

```
module top(                    //Input/output interface of top module
    input  wire clk,           // 100MHz clk
    input  wire [7:0] key,     // input 3
    input  wire uart_rx,       // data received on UART RX
    output wire uart_tx,       // data sent on UART TX
    output reg [7:0] led,      // display the data received on UART RX
    input uart_tx_rst_n,       // uart tx reset, low level valid
    input uart_rx_rst_n,       //uart rx reset, low level valid
    input send_one,            // when pressed, send the data on UART TX port
    output uart_tx_work,       //LED used to display whether UART TX is working
    output uart_rx_work        //LED used to display whether UART RX is working
);
```

A fragment of the constraint file(Based on EGO1):

```
set_property PACKAGE_PIN P17 [get_ports clk]
set_property PACKAGE_PIN R11 [get_ports send_one]
set_property PACKAGE_PIN T3 [get_ports uart_tx_rst_n]
set_property PACKAGE_PIN T5 [get_ports uart_rx_rst_n]
set_property PACKAGE_PIN M1 [get_ports uart_tx_work]
set_property PACKAGE_PIN K3 [get_ports uart_rx_work]
set_property PACKAGE_PIN T4 [get_ports uart_tx]
set_property PACKAGE_PIN N5 [get_ports uart_rx]
```

WANGW6@SUSTECH.EDU.CN

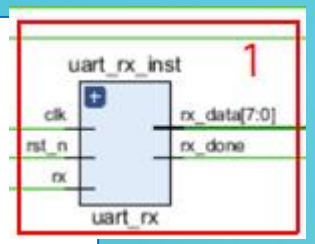
UART DEMO (3) - STRUCTURE DESIGN(1)



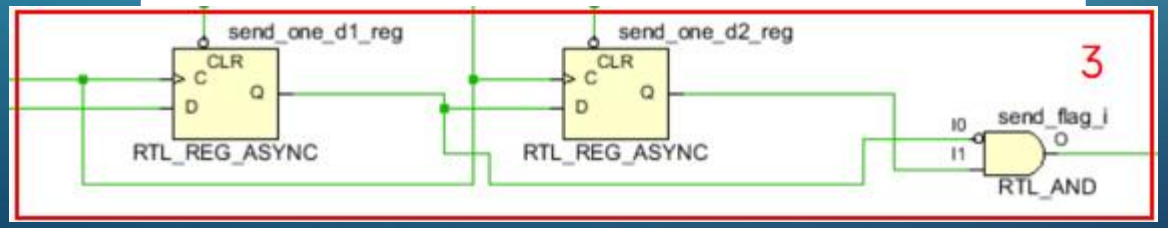
- Submodule 1 in the circuit, used to extract the valid data received on UART RX.
- Submodule 2 in the circuit, used to display the data(received on UART RX) on the LED group after data collection is completed.
- Submodule 3 in the circuit, used to detect whether the confirmation key is pressed.
- Submodule 4 in the circuit, used to send the data(set by the switches) after the confirmation key is pressed through UART TX.

UART DEMO (4) - STRUCTURE DESIGN(2)

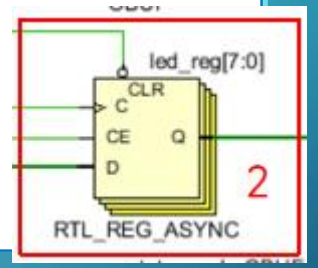
```
// ===== UART RX =====
    wire [7:0] rx_data;
    wire rx_done;


assign uart_rx_work = uart_rx_rst_n;
    uart_rx #(
        .CLK_FREQ(100_000_000),
        .BAUD_RATE(115200)
    ) uart_rx_inst (
        .clk(clk),
        .rst_n( uart_rx_rst_n  ),
        .rx(uart_rx),
        .rx_data(rx_data),
        .rx_done(rx_done)
    );
```



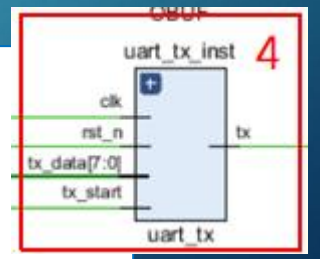

```
// = =Check “send_one” button has been pressed or not ==
 reg send_one_d1, send_one_d2;
 always @(posedge clk or negedge uart_tx_rst_n) begin
     if(!uart_tx_rst_n) begin
        send_one_d1 <= 1'b0;
        send_one_d2 <= 1'b0;
     end
     else begin
        send_one_d1 <= send_one;
        send_one_d2 <= send_one_d1;
     end
 end
 wire send_flag;
 assign send_flag = ~send_one_d1 & send_one_d2;
```




```
// ===== LED display the received data =====
    always @(posedge clk or negedge uart_rx_rst_n) begin
        if (!uart_rx_rst_n)
            led <= 8'b0;
        else if (rx_done)
            led <= rx_data;
    end

endmodule
```

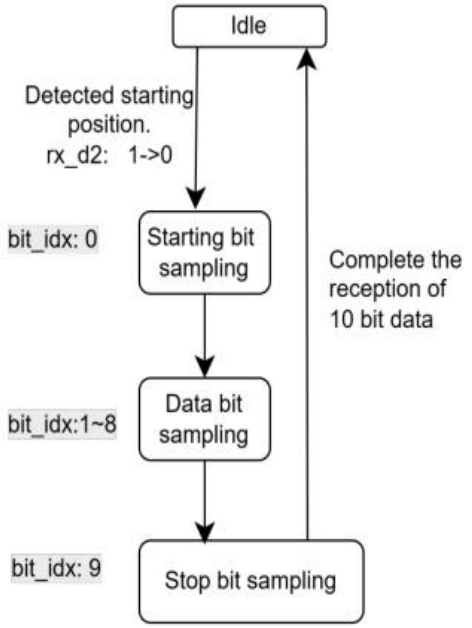



```
// ===== UART TX =====
uart_tx #(
        .CLK_FREQ(100_000_000),
        .BAUD_RATE(115200)
    ) uart_tx_inst (
        .clk(clk),
        .rst_n( uart_tx_rst_n),
        .tx_start(send_flag),
        .tx_data(key),
        .tx(uart_tx),
        .tx_busy(tx_busy)
    );
```



WANGW6@SUSTECH.EDU.CN

UART (5)- UART RX




```
module uart_rx #(
    parameter CLK_FREQ = 100_000_000,
    parameter BAUD_RATE = 115200
)(
    input wire clk,
    input wire rst_n,
    input wire rx,              // Serial input signal
    output reg [7:0] rx_data,  // the  byte received
    output reg rx_done    // Data reception completion flag
);

    localparam BAUD_DIV = CLK_FREQ / BAUD_RATE;

    reg [15:0] baud_cnt;
    reg [3:0] bit_idx;
    reg [9:0] rx_shift;
    reg rx_busy;
    reg rx_d1, rx_d2;
```

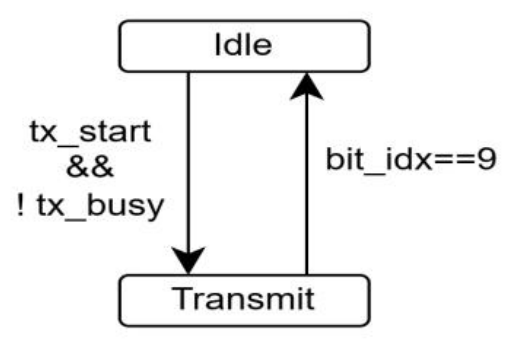
```
always @(posedge clk) begin
        rx_d1 <= rx;
        rx_d2 <= rx_d1;
        end
```

```
always @(posedge clk or negedge rst_n) begin
        if ( ! rst_n ) begin
            baud_cnt <= 0;
            bit_idx <= 0;
            rx_busy <= 0;
            rx_done <= 0;
            rx_data <= 8'b0;
        end else begin
            rx_done <= 0;
            if  ( ! rx_busy ) begin
                if (rx_d2 == 0) begin            // Detected starting position
                    rx_busy <= 1;
                    baud_cnt <= BAUD_DIV / 2;        // Align to the center of the data
                    bit_idx <= 0;
                end
            end else begin
                if (baud_cnt == BAUD_DIV - 1) begin
                    baud_cnt <= 0;
                    bit_idx <= bit_idx + 1;
                    case (bit_idx)
                        0:  ;                    // starting bit
                        1,2,3,4,5,6,7,8: rx_shift[bit_idx-1] <= rx_d2;
                        9: begin
                            rx_busy <= 0;
                            rx_data <= rx_shift[7:0];
                            rx_done <= 1;
                        end
                    endcase
                end else
                    baud_cnt <= baud_cnt + 1;
            end
        end
    end
endmodule
```

start bit(1)	data bits(8)	stop bit(1)

WANGW6@SUSTECH.EDU.CN

UART (6)- UART TX




```
module uart_tx #(
    parameter CLK_FREQ = 100000000,
    parameter BAUD_RATE = 115200
)(
    input wire clk,
    input wire rst_n,
    input wire tx_start,          // Trigger sending signal
    input wire [7:0] tx_data,     // The byte to be sent
    output reg tx,                // Serial output cable
    output reg tx_busy            // flag of Sending
);

localparam BAUD_DIV = CLK_FREQ / BAUD_RATE;

    reg [15:0] baud_cnt;
    reg [3:0] bit_idx;
    reg [9:0] tx_shift;
```

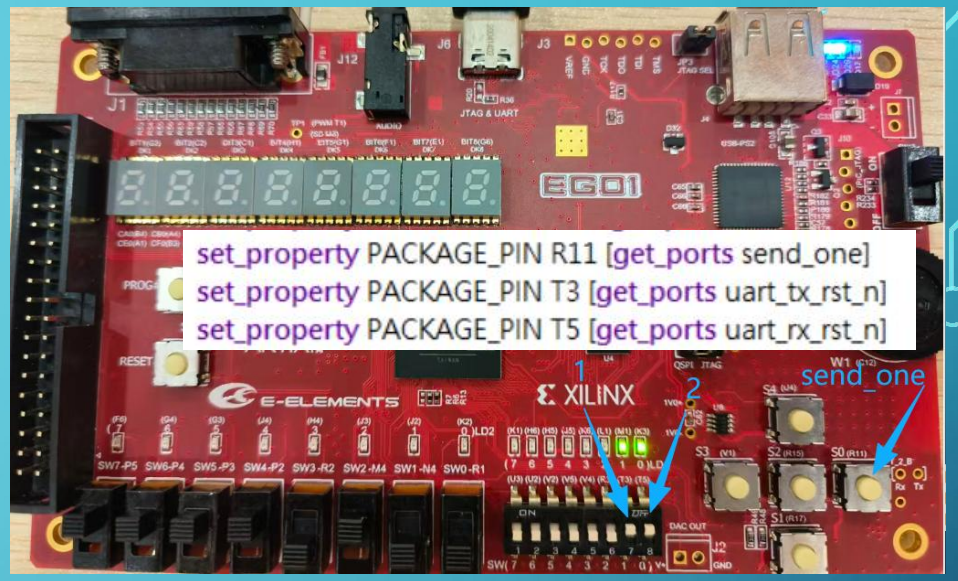
```
always @(posedge clk or negedge rst_n) begin
        if( !rst_n ) begin
            baud_cnt <= 0;
            bit_idx <= 0;
            tx_shift <= 10'b1111111111;
            tx <= 1'b1;
            tx_busy <= 1'b0;
        end else begin
            if( tx_start && ! tx_busy) begin
                // data:   stop bit(1),  data(LSB first) , starting bit(0)
                tx_shift <= {1'b1, tx_data, 1'b0};
                tx_busy <= 1'b1;
                baud_cnt <= 0;
                bit_idx <= 0;
            end else if(tx_busy) begin
                if(baud_cnt < BAUD_DIV - 1) begin
                    baud_cnt <= baud_cnt + 1;
                end else begin
                    baud_cnt <= 0;
                    tx <= tx_shift[0];
                    tx_shift <= {1'b1, tx_shift[9:1]};  // right shift 1 bit
                    bit_idx <= bit_idx + 1;
                    if(bit_idx == 9) begin
                        tx_busy <= 1'b0;
                    end
                end
            end
        end
    end
endmodule
```

start bit(1)	data bits(8)	stop bit(1)

UART(7)- TESTING(1)

Get ready on EGO1.

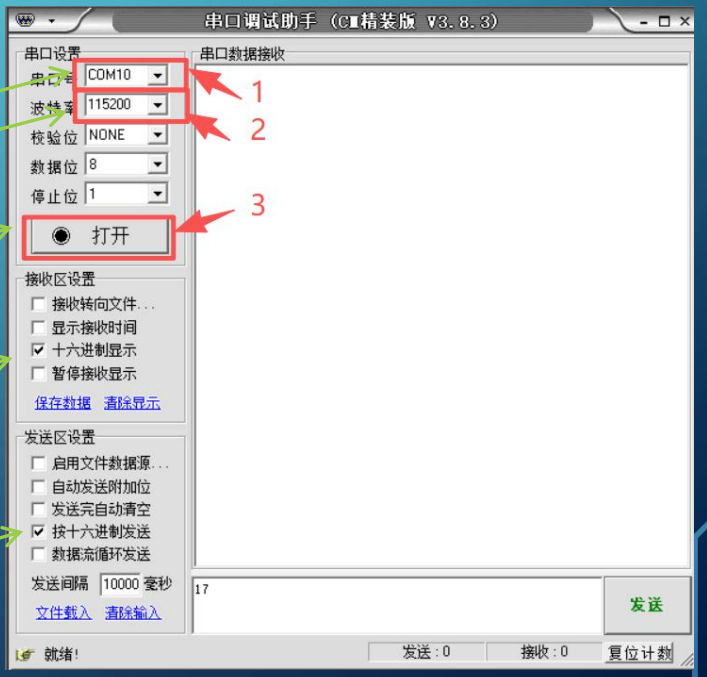
- Step1. Program the bitstream of the circuit to the FPGA chip on EGO1.
- Step2. turn on swith1 and switch2(marked by arrows 1 and 2 in the figure on the right) on the EGO1 to make uart rx and uart rx work in the circuit.



Get ready on "UART communication software"(串口调试助手) which works on the PC.

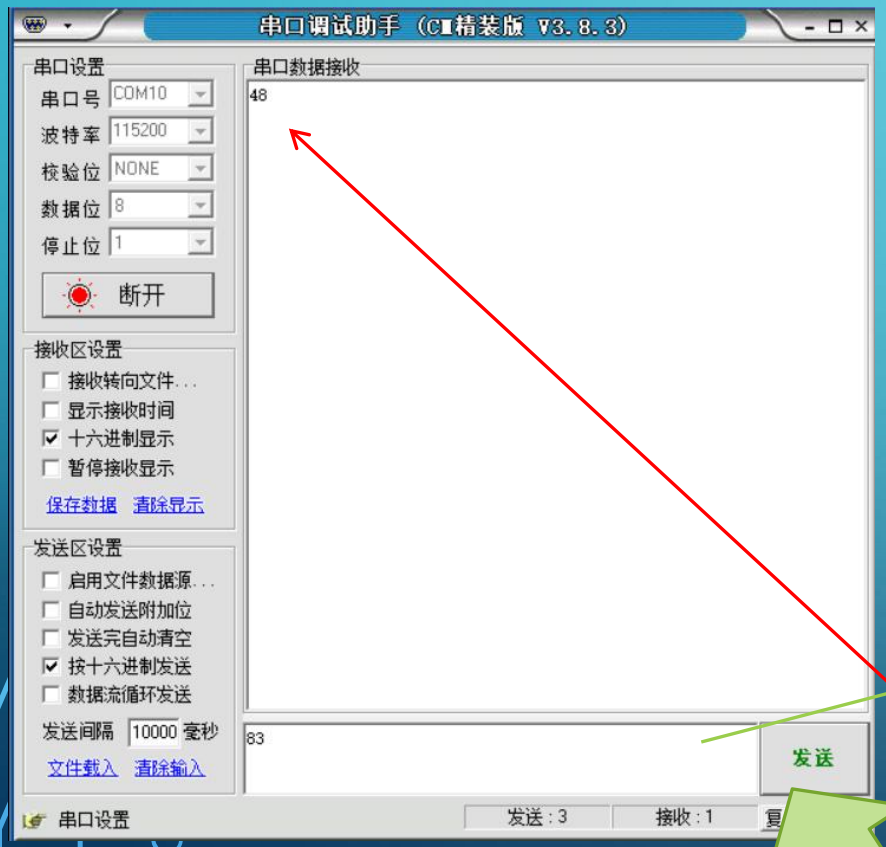
- Open UART communication software(串口调试助手) on PC
- Set serial port(串口设置)
  - ✓ Choose the serial port (串口号) which connects to Uart of EGO1
  - ✓ Set Baud rate(波特率) : 115200 (same rate in verilog)
  - ✓ Set check bits(校验位)： None
  - ✓ Set the bitwidth of data (数据位)： 8
  - ✓ Set the bitwidth of stop bits(停止位) : 1
- Click on "打开"

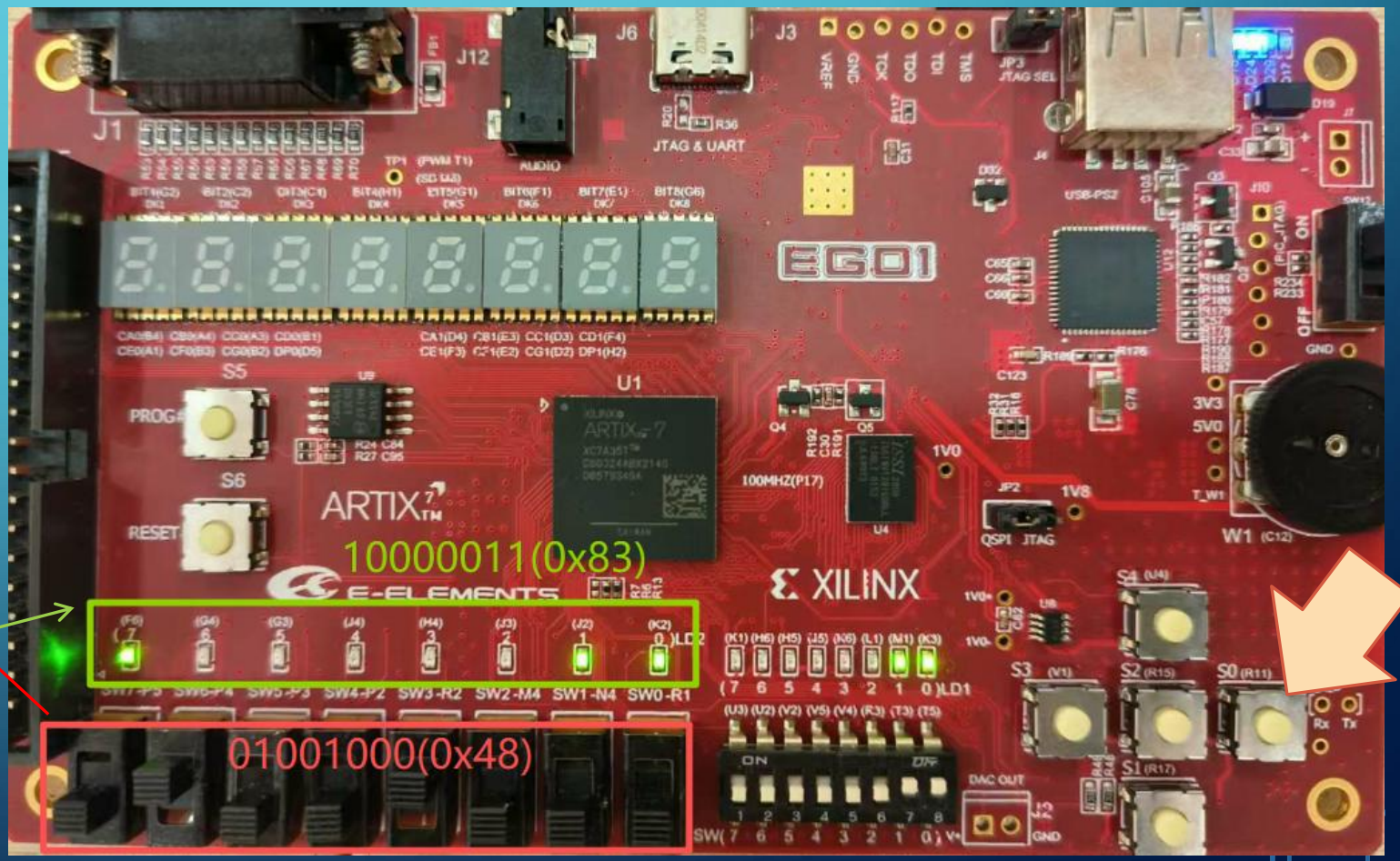
Optional:
- receive data,show it in hexadecimal
- Send data which is in hexadecimal



UART(8)- TESTING(2)

- **UART TX testing:** 1) set the sending data 01001011(48 in hexdecimal) on the switches ， 2)pressed confirm button. Then “48” should be displayed in the receiving window(串口数据接收) of “UART communication software”(串口调试助手).
- **UART RX testing:** input 83(which is in hexdecimal) in sending window, click on “发送”. Then 10000011(binary of 0x83) should be displayed on the led group.

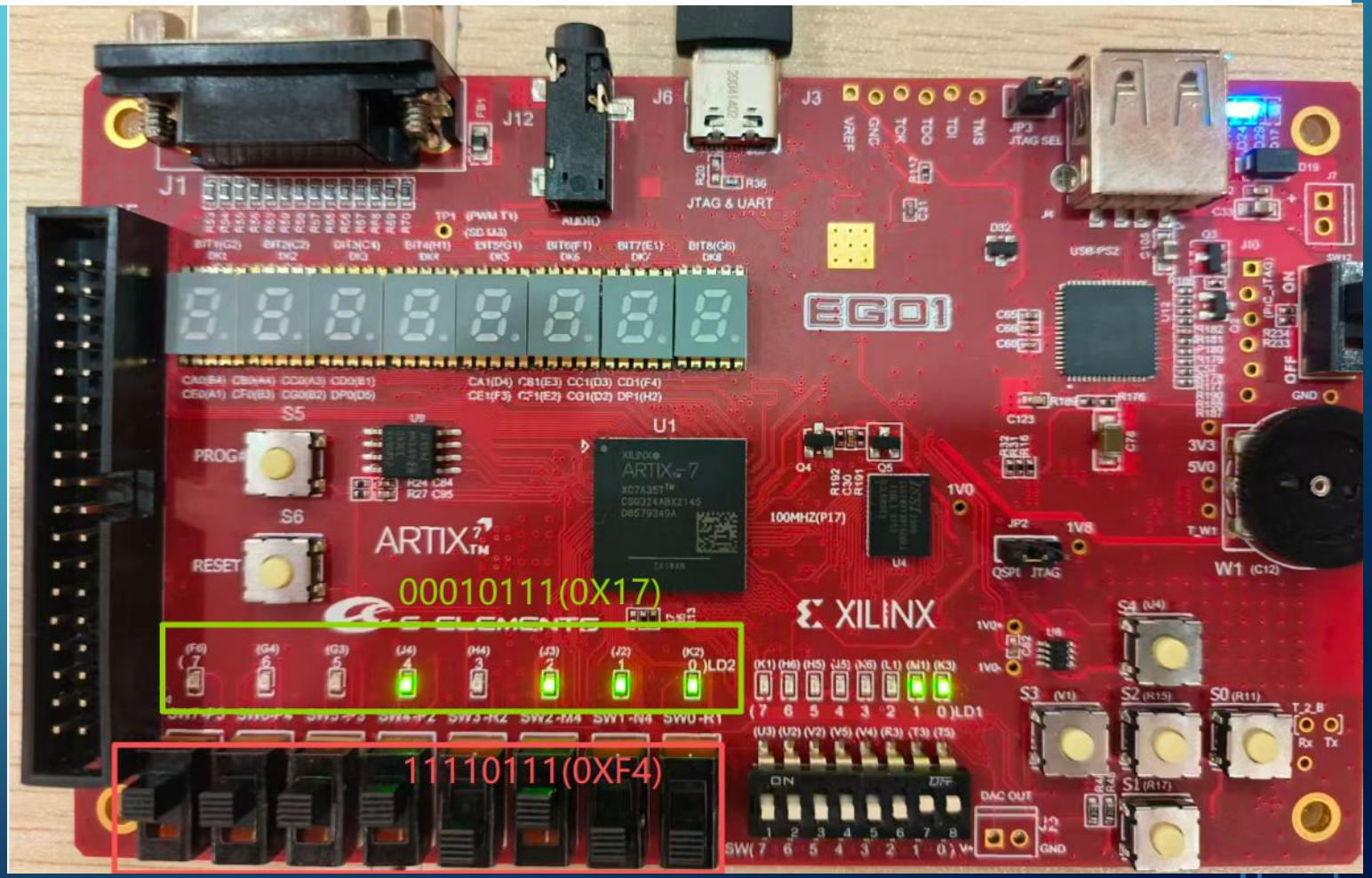




WANGW6@SUSTECH.EDU.CN

UART(9)- TESTING(3)

- **UART TX testing:** 1) set the sending data 11110011(F4 in hexdecimal) on the switches， 2)pressed confirm button. Then “F4” should be displayed in the receiving window(串口数据接收) of “UART communication software”(串口调试助手).
- **UART RX testing:** input 17(which is in hexdecimal) in sending window, click on “发送”. Then 00010111(binary of 0x17) should be displayed on the led group.



WANGW6@SUSTECH.EDU.CN

TOPIC

- 1. Verilog
  - parameter used at module level
- 2. Uart demo
- **3. Debug by simulation**

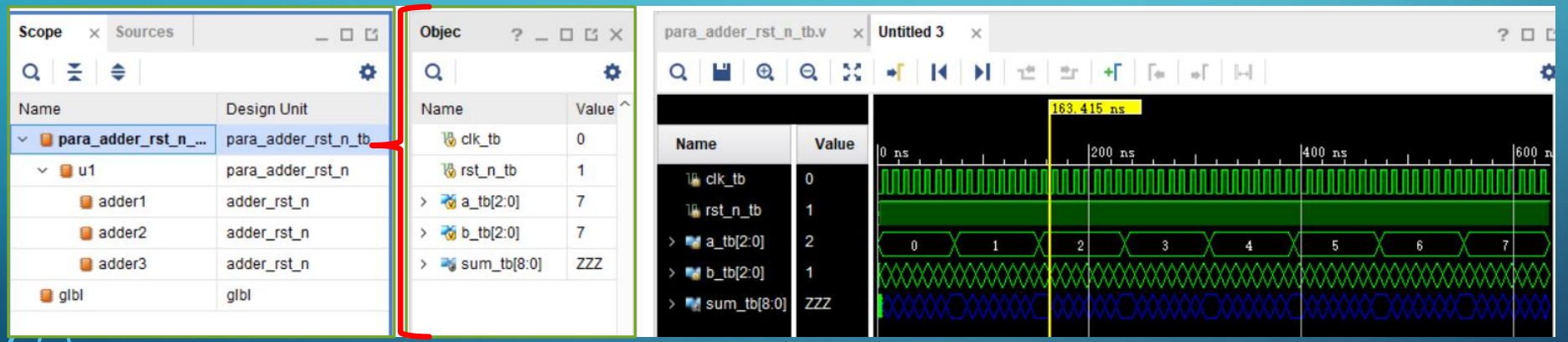
WANGW6@SUSTECH.EDU.CN

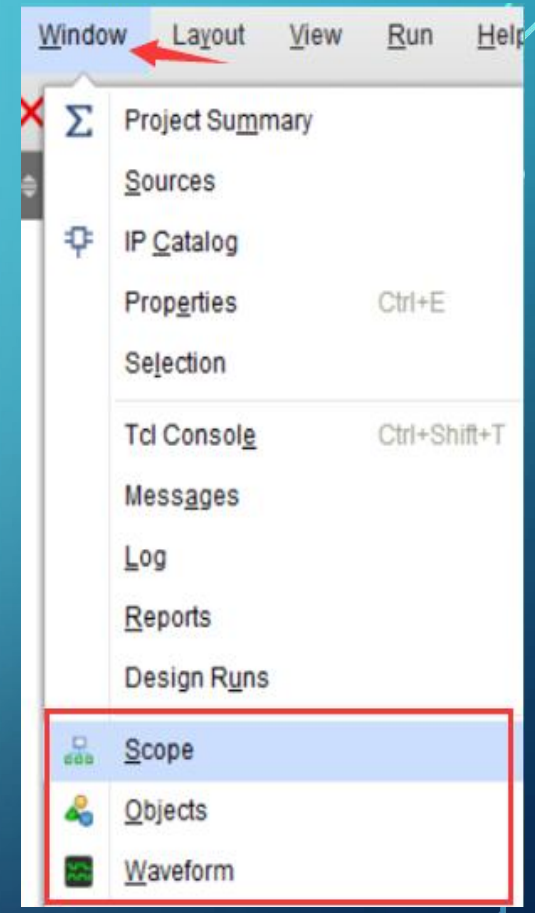
DEBUG BY SIMULATION - USEFULL WINDOWS

After simulation, Vivado will invoke three windows related to the simulation results:

- ✓ **waveform** window displays **all data information in the testbench module by default**.
- ✓ **Scope** window displays **module information**.
- ✓ **Object** window displays **the port and internal signal information of the module which is specified in the scope window**.

The information in these three windows can help locate behavioral issues in the module.

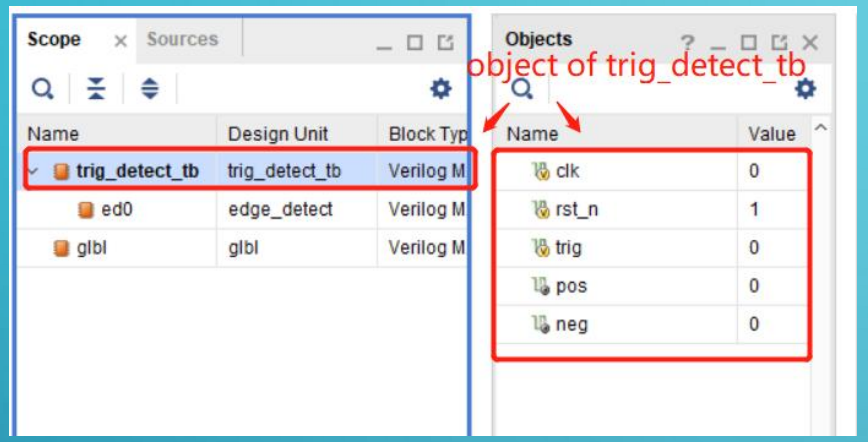


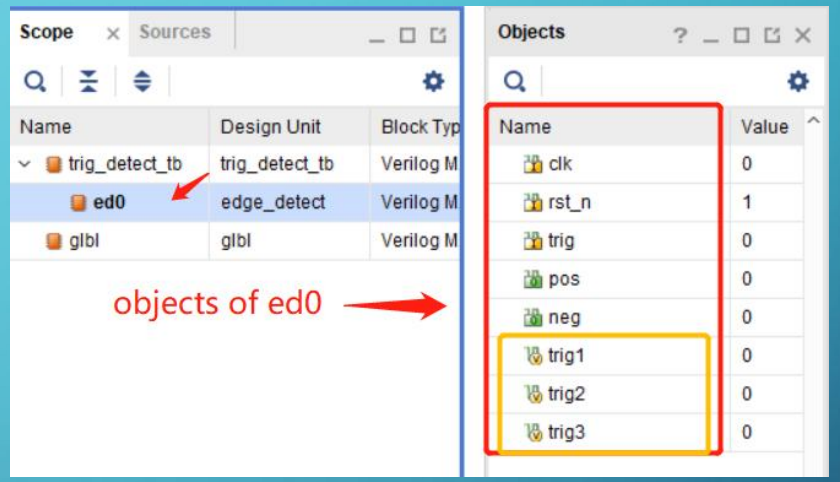


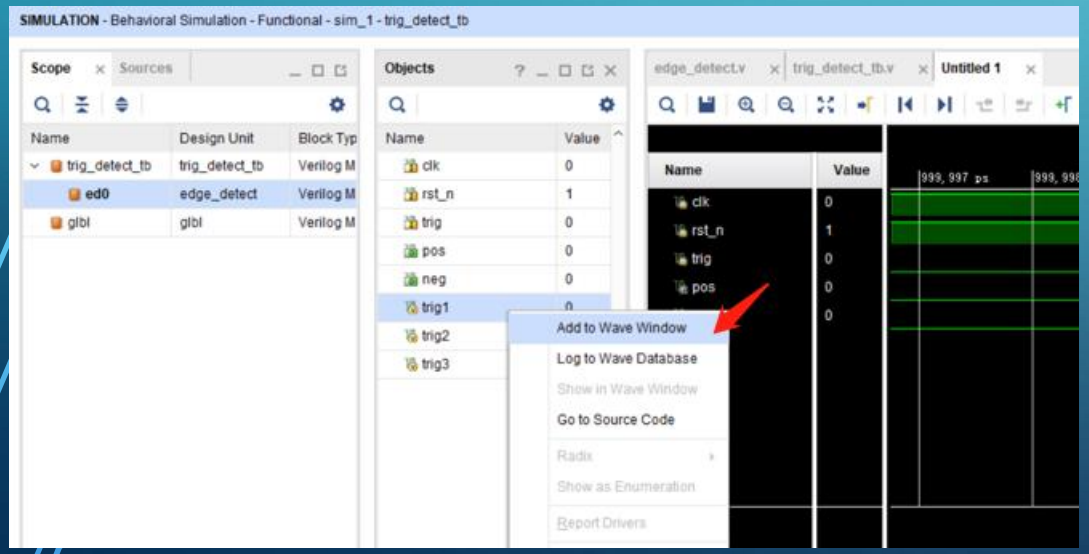
WANGW6@SUSTECH.EDU.CN

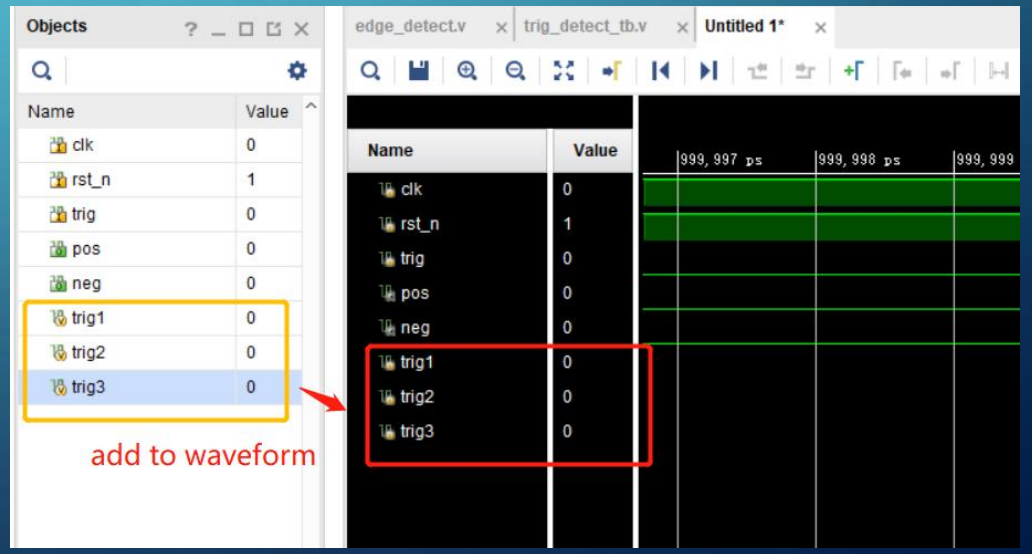
DEBUG BY SIMULATION -STEP1

- Step1: add intral signal to waveform if needed



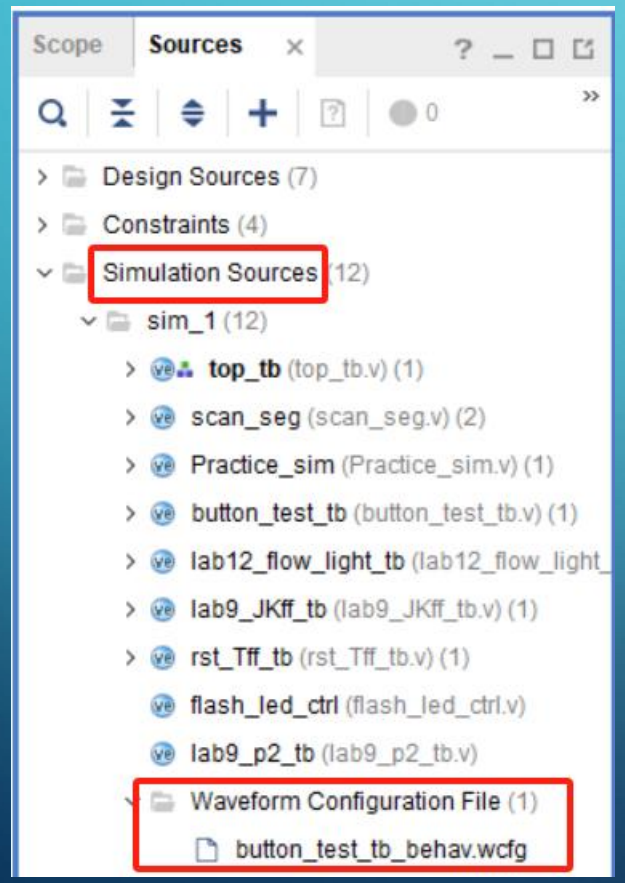






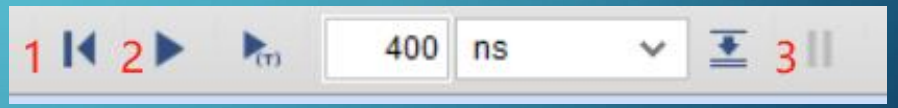
DEBUG BY SIMULATION - STEP2

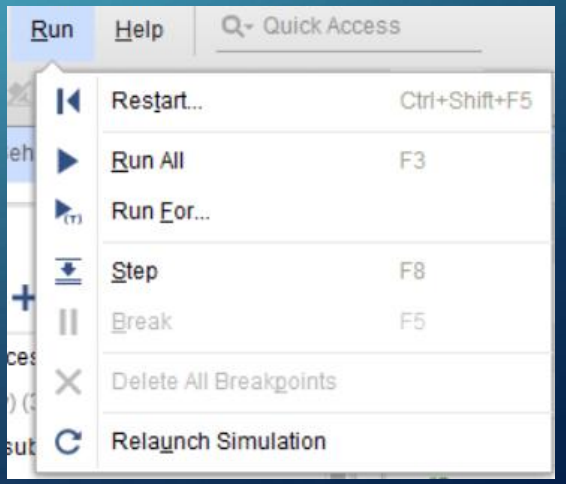
- Step2: run the simulation again
  - **method 1**: save the waveform as wcfg file and add it to the project, the simulation will rerun as the wcfg file specified.



- method 2: restart the simulation without save the modification on the waveform.
  - Restart -> specify the simulation time -> Run For the specified time

  - Restart -> Run All , then “Break” as needed.



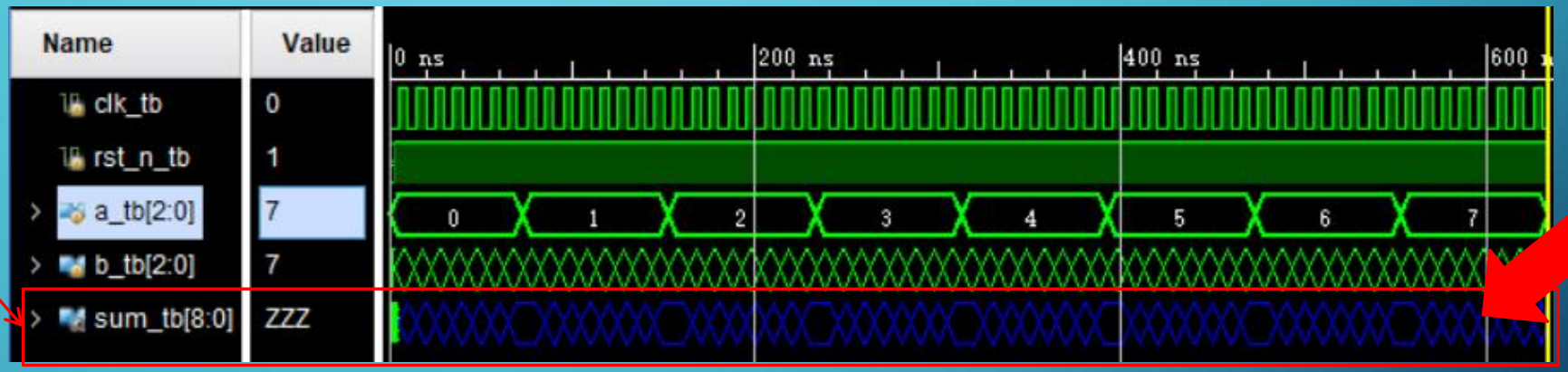


DEBUG BY SIMULATION - DEMO

The simulation on **para_adder_rst_n_tb** show that the output signal “sum_tb” is incorrect(**unexpected Z appears**).

para_adder_rst_n_tb is the testbench of **para_adder_rst_n.** In **para_adder_rst_n**, three **adder_rst_n** are instanced. The **adder_rst_n has an asynchronous reset signal, outputs 0 when the reset signal is 0, and outputs the sum of the two input signals at the current time when the reset signal is 1 and the clock is rising**.

```
module para_adder_rst_n_tb( );
  reg clk_tb, rst_n_tb;
  reg [2:0] a_tb, b_tb;
  wire [8:0] sum_tb;
  para_adder_rst_n u1(clk_tb, rst_n_tb, a_tb, b_tb, sum_tb);
  initial begin
  clk_tb = 1'b0;
  forever #5 clk_tb = ~clk_tb;
  end
  initial begin
  rst_n_tb = 1'b0;
  #1  rst_n_tb = 1'b1;
  end
  initial begin
  {a_tb, b_tb} = 6'b0;
  #3 {a_tb, b_tb} = 6'b1;
  repeat(62) #10 {a_tb, b_tb} = {a_tb, b_tb} + 1;
  #10 $finish;
  end
endmodule
```

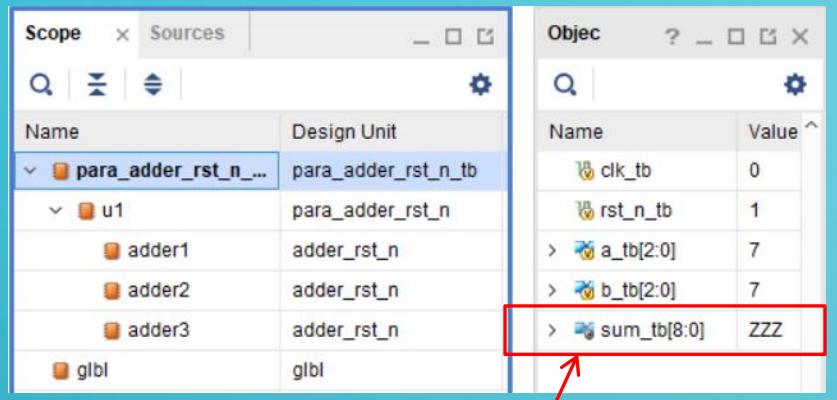


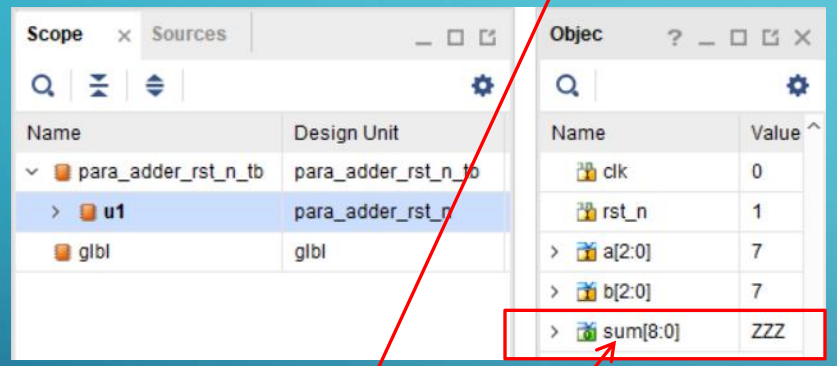

```
module para_adder_rst_n(input clk, rst_n, input [2:0] a, b, output [8:0] sum  );

  adder_rst_n #(.WIDTH(1)) adder1 (
    .clk(clk), .rst_n(rst_n), .a(a[0]), .b(b[0]), .sum(sum[1:0])  );

  adder_rst_n adder2 (
    .clk(clk), .rst_n(rst_n),  .a(a[1:0]), .b(b[1:0]), .sum(sum[4:2])  );

  adder_rst_n #(.WIDTH(3))adder3 (
    .clk(clk), .rst_n(rst_n), .a(a[2:0]), .b(b[2:0]), .sum(sum[8:5])   );
endmodule
```

```
module  adder_rst_n#(parameter WIDTH = 2) (
  input rst_n,
  input clk,
  input [WIDTH-1:0] a,
  input [WIDTH-1:0] b,
  output [WIDTH:0] sum
);
  reg sum;
  always @(posedge clk, negedge rst_n)
    if( ! rst_n)
      sum <= 0;
    else
      sum <= a + b;
endmodule
```

WANGW6@SUSTECH.EDU.CN

DATA FLOW TRACING(1)





"sum_tb" of "para_adder_rst_n_tb" is from "sum" of "u1" which is a instance of "para_adder_rst_n".

Both "sum_tb" and "sum" are ZZZ !

"sum" of "para_adder_rst_n" is grouped by "sum" of "adder1", "adder2" and ", "adder3".

```
module para_adder_rst_n(input clk, rst_n, input [2:0] a, b, output [8:0] sum );

  adder_rst_n #(.WIDTH(1)) adder1 (
    .clk(clk), .rst_n(rst_n), .a(a[0]), .b(b[0]), .sum(sum[1:0]) );

  adder_rst_n adder2 (
    .clk(clk), .rst_n(rst_n),  .a(a[1:0]), .b(b[1:0]), .sum(sum[4:2]) );

  adder_rst_n #(.WIDTH(3))adder3 (
    .clk(clk), .rst_n(rst_n), .a(a[2:0]), .b(b[2:0]), .sum(sum[8:5])  );
endmodule
```

It seems that there is no problem with the port connection in the code of "para_adder_rst_n" ,

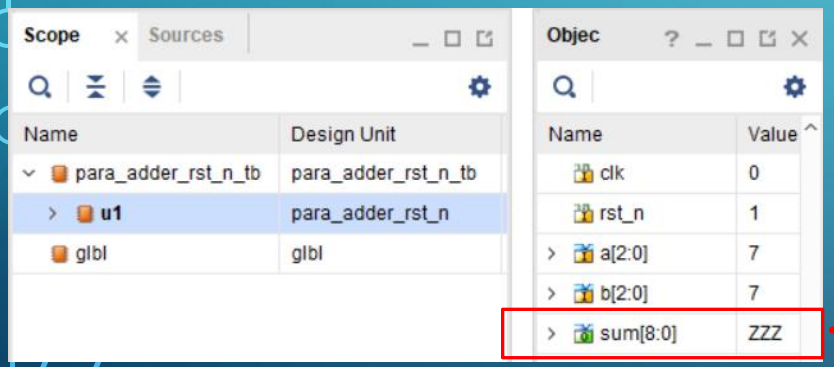
so where is the problem ?

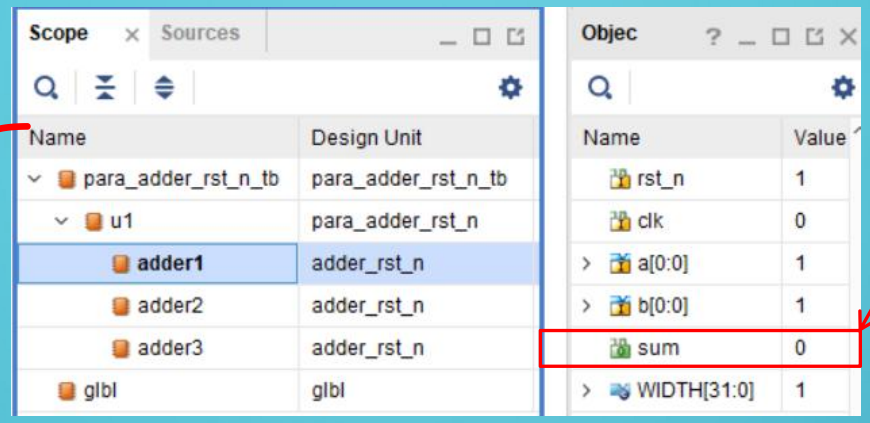
WANGW6@SUSTECH.EDU.CN

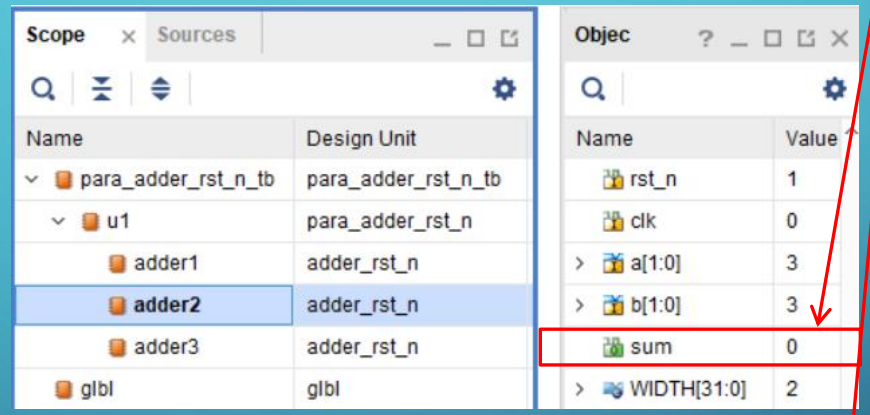
DATA FLOW TRACING(2)

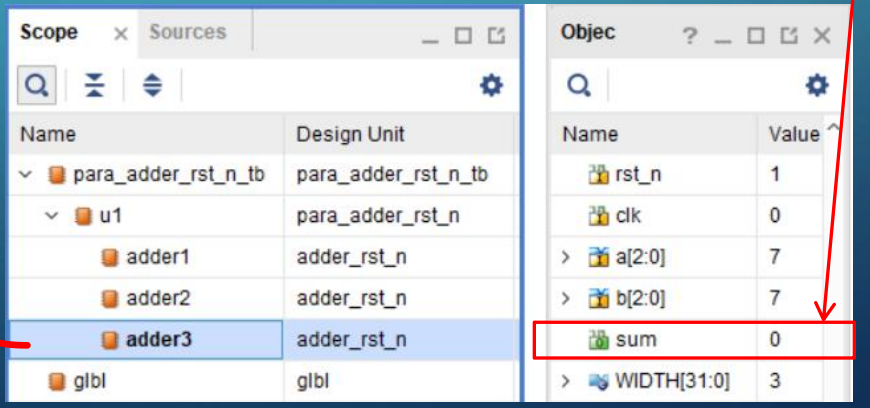
“sum” of “para_adder_rst_n” is grouped by “sum“ of ”adder1“, ”adder2“ and ”, “adder3“.

```
module para_adder_rst_n(input clk, rst_n, input [2:0] a, b, output [8:0] sum );

  adder_rst_n #(.WIDTH(1)) adder1 (
    .clk(clk), .rst_n(rst_n), .a(a[0]), .b(b[0]), .sum(sum[1:0]) );

  adder_rst_n adder2 (
    .clk(clk), .rst_n(rst_n),  .a(a[1:0]), .b(b[1:0]), .sum(sum[4:2]) );

  adder_rst_n #(.WIDTH(3))adder3 (
    .clk(clk), .rst_n(rst_n), .a(a[2:0]), .b(b[2:0]), .sum(sum[8:5])  );
endmodule
```









Something wrong with the bitwidth of “sum” of “adder_rst_n” !

Why are the bit widths of the output port “sum” of three submodules(adder1, adder2 and adder3) all 1 bit ?

Fix the problem in Practice3.

```
module  adder_rst_n#(parameter WIDTH = 2) (
   input rst_n,
   input clk,
   input [WIDTH-1:0] a,
   input [WIDTH-1:0] b,
   output [WIDTH:0] sum
);
   reg sum;
   always @(posedge clk, negedge rst_n)
      if( ! rst_n)
         sum <= 0;
      else
         sum <= a + b;
endmodule
```

?

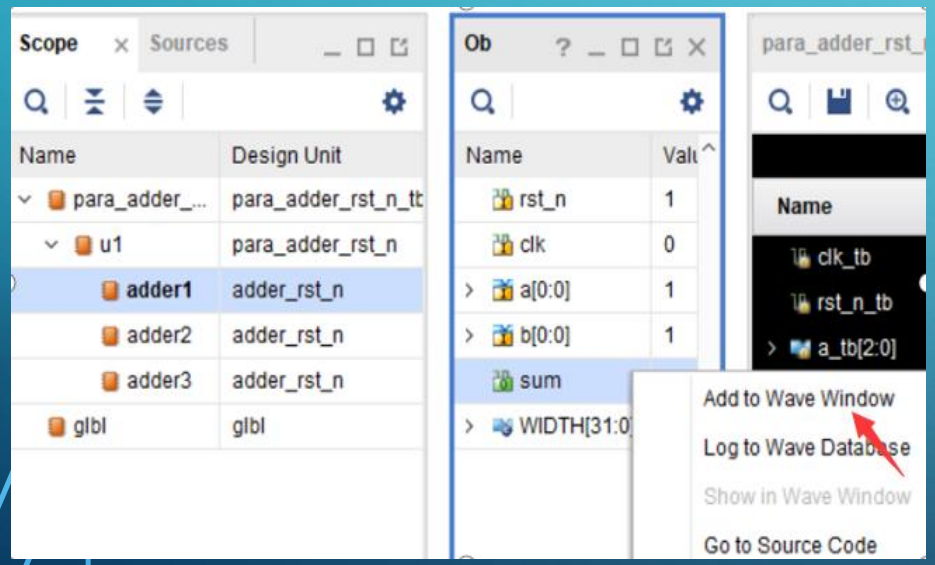
WANGW6@SUSTECH.EDU.CN

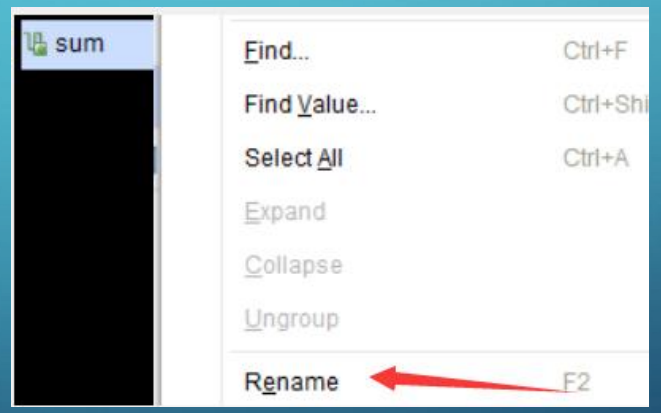
ADD VARIABLE TO WAVEFORM

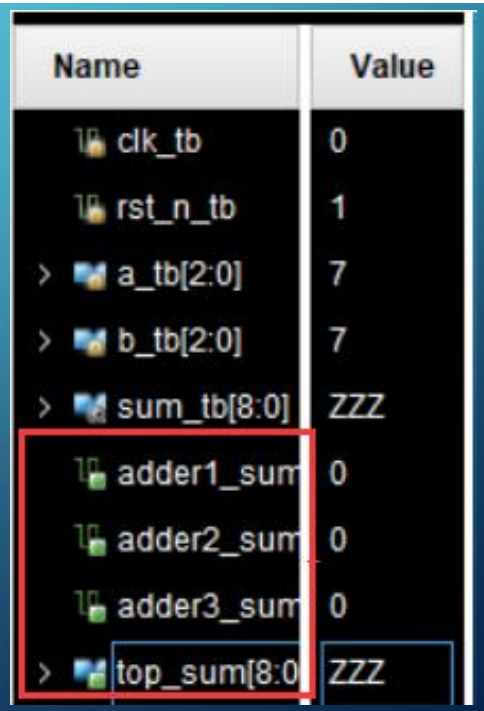
Variable(Ports or signals) in the module can be added to the waveform window:

- **1st: select module in Scope window**
- **2nd:** in "**Object**" window, **select** the variable(Ports or signals) and **right-click** on it, then select "**Add to Wave Window**

TIPS: The variable can be **renamed** in the waveform window (for easy distinction if necessary), and their changes can be observed through **rerun simulation** during the simulation process. In the following demo, "sum" of "adder1", "adder2", "adder3" and "u1" are added to the wavewindow, and are renamed as "adder1_sum", "adder2_sum", "adder3_sum" and "top_sum".



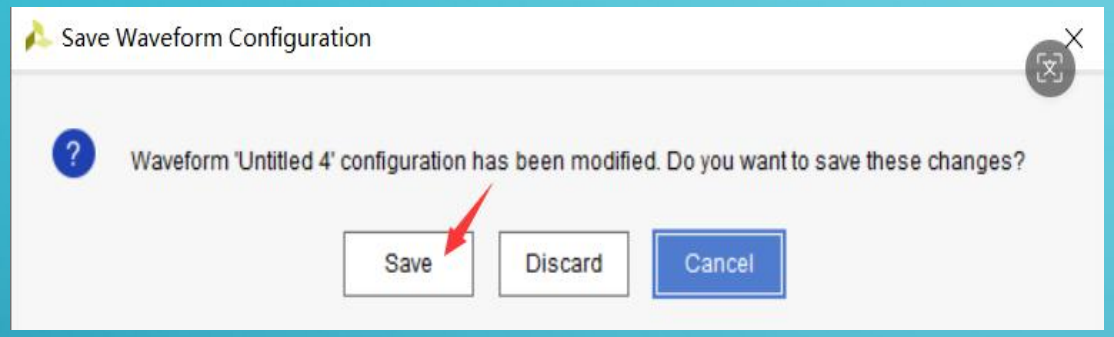


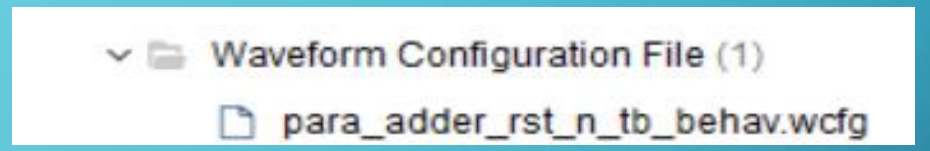


WANGW6@SUSTECH.EDU.CN

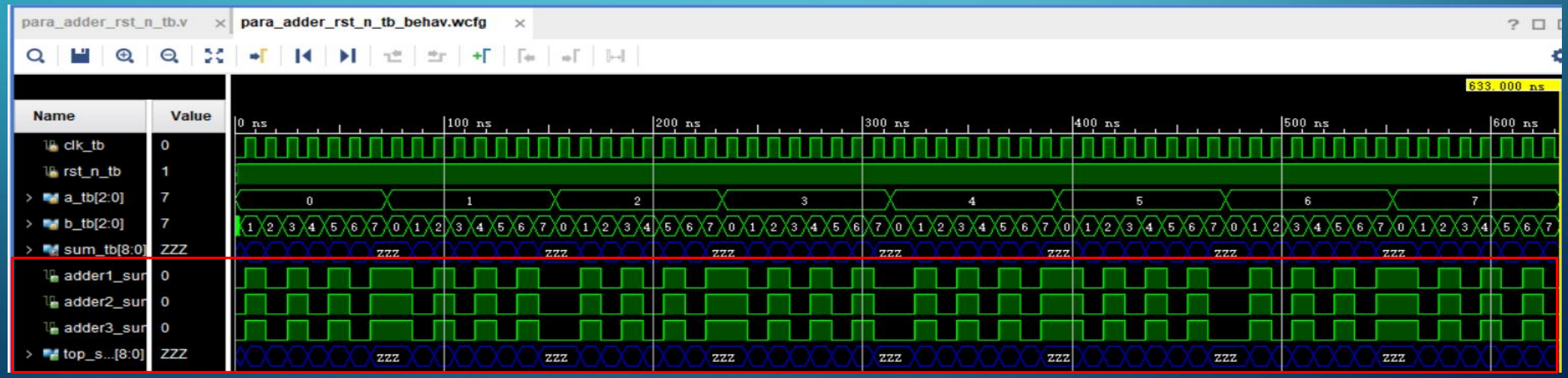
WAVEFORM CONFIGRATION FILE(SAVE, ADD, DISABLE)

When rerun the simulation after add/remove variable to/from waveform, the simulator will remind you whether to save the new waveform changes. Select "**Save**", and the waveform configuration file will be **saved and added to the project directory**. The configuration file would be in the "**Waveform Configuration File**" in the sources window of Vivado. **Subsequent simulations of the project will be based on this waveform configuration file**.





TIPS: If you do not need this waveform configuration file, you can right-click on it and select **disable**. This file will render the simulation invalid.



WANGW6@SUSTECH.EDU.CN

PRACTICE1 (TEAM WORK)

- 1. Analyze the interaction between the UART module and the project, and determine the interaction relationship between each sub module, top-level module, and Uart communication module in the structured design.

- 2. Develop project work specifications (code specifications, test specifications, code submission specifications), plan test cases.

- 3. Make work plans.

WANGW6@SUSTECH.EDU.CN

PRACTICE2

1. Program the UART circuit provided in this experiment to FPGA chip of EGO1.

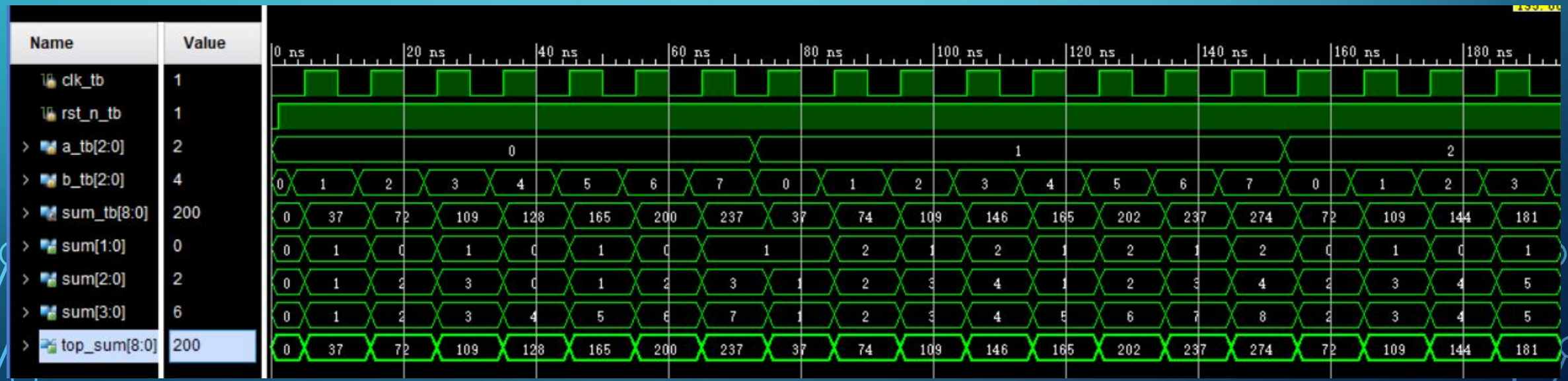
2. set the ASCII code of character '*' , press comformation button on the EGO1, sent the data to the "UART communication software"(串口调试助手).

3. Character * should be displayed in the receiving window of the "UART communication software"(串口调试助手). (as the picture on the right)

PRACTICE3

Resolve the code issues in the debug demo(page20-24), and then re-run the simulation.

The simulation waveform should be as shown in the following figure



WANGW6@SUSTECH.EDU.CN